

ENTITY RADAR

TACTICAL DASHBOARD

Dokumentasi Teknis & Referensi API

Spesifikasi Arsitektur, Panduan Eksekusi, dan Alur Kerja Aplikasi

Production Live: map.portofoliorifky.my.id

Daftar Isi

- Daftar Isi..... 2
- 1. Ikhtisar Proyek 3
 - 1.1 Tumpukan Teknologi & Alasan Pemilihan 3
- 2. Panduan Instalasi & Menjalankan Aplikasi 3
 - 2.1 Inisialisasi Database..... 3
 - 2.2 Menjalankan Backend API 3
 - 2.3 Menjalankan Frontend Klien 3
- 3. Spesifikasi RESTful API 4
- 4. Alur Kerja Aplikasi (User Workflow) 6
 - 4.1 Tampilan Utama (Tactical Dashboard) 6
 - 4.2 Menambahkan Entitas Baru (Geofencing Click-to-Pick) 6
 - 4.3 Mengedit Data dan Sistem Notifikasi..... 7
- 5. Transparansi Keterlibatan AI 8
 - 5.1 Ruang Lingkup Bantuan AI (Peningkatan Produktivitas) 8
 - 5.2 Kemandirian Pengembang (Batas Intervensi AI)..... 8
- 6. Keterbatasan Sistem & Evaluasi Fungsional 9
 - 6.1 Akurasi Radius Spasial (Geofencing)..... 9
 - 6.2 Komunikasi Dupleks (Real-Time Sync)..... 9
 - 6.3 Resiliensi Mode Luring (Offline) 9

1. Ikhtisar Proyek

Entity Radar adalah aplikasi pemetaan entitas geospasial full-stack dengan antarmuka bergaya command center. Sistem ini memungkinkan pengguna memantau, menambahkan, dan mengedit data entitas geografis lengkap dengan koordinatnya secara interaktif, langsung dari kanvas peta.

1.1 Tumpukan Teknologi & Alasan Pemilihan

Lapisan	Pilihan Teknologi	Alasan Pemilihan
Backend	Go (Golang) + Gin	Dipilih untuk menghasilkan RESTful API berkinerja tinggi. Waktu kompilasi cepat dan penggunaan memori rendah, ideal untuk skalabilitas mikroservis.
Database	PostgreSQL + GORM	PostgreSQL menyediakan keandalan relasional tingkat enterprise. GORM bertindak sebagai ORM untuk mencegah kerentanan SQL injection dan memfasilitasi migrasi skema otomatis.
Frontend	React + TypeScript + Vite	TypeScript memberlakukan validasi tipe data statis untuk mencegah galat saat runtime. Vite menawarkan waktu build instan dibandingkan Webpack tradisional.
UI / State	Tailwind + Zustand + Leaflet	Tailwind memungkinkan gaya antarmuka kustom (glassmorphism) secara efisien. Zustand mengelola state global tanpa boilerplate rumit. Leaflet meminimalkan beban rendering peta dibandingkan mesin WebGL.

2. Panduan Instalasi & Menjalankan Aplikasi

Prasyarat: pastikan sistem sudah memiliki Go (v1.20+), Node.js (v18+), dan PostgreSQL terpasang.

2.1 Inisialisasi Database

Buat pangkalan data baru di PostgreSQL. Skema tabel akan dibuat secara otomatis saat backend pertama kali dijalankan.

```
CREATE DATABASE entity_map_db;
```

2.2 Menjalankan Backend API

Masuk ke direktori backend, pastikan kredensial database pada main.go atau .env sudah sesuai, kemudian jalankan layanan.

```
cd backend
go mod tidy
go run main.go
```

Layanan akan berjalan di porta :8085 dengan kebijakan CORS yang telah dikonfigurasi.

2.3 Menjalankan Frontend Klien

Masuk ke direktori frontend dan jalankan peladen pengembangan. Target API secara bawaan menunjuk ke server produksi.

```
cd frontend
npm install
npm run dev
```

3. Spesifikasi RESTful API

Base URL Produksi:

```
https://mapapi.portofoliorifky.my.id/api/entities
```

GET

/

Mengambil seluruh data entitas geospasial yang terdaftar.

Respons sukses (200 OK):

```
[
  {
    "id": 1,
    "name": "Sektor Alpha",
    "type": "military",
    "lat": -6.914744,
    "lng": 107.609810,
    "status": "Active",
    "description": "Pusat komando utama"
  }
]
```

POST

/

Menambahkan entitas baru ke dalam pangkalan data.

Format payload permintaan (JSON):

```
{
  "name": "Pos Pengamatan B",
  "type": "civilian",
  "lat": -6.920000,
  "lng": 107.610000,
  "status": "Under Maintenance",
  "description": "Pemeliharaan rutin jaringan"
}
```

PUT**/:id**

Memperbarui data entitas yang ada berdasarkan ID spesifik.

Respons sukses (200 OK): mengembalikan objek entitas yang telah diperbarui.

DELETE**/:id**

Menghapus entitas secara permanen dari pangkalan data.

Respons sukses (200 OK):

```
{ "message": "Entity deleted" }
```

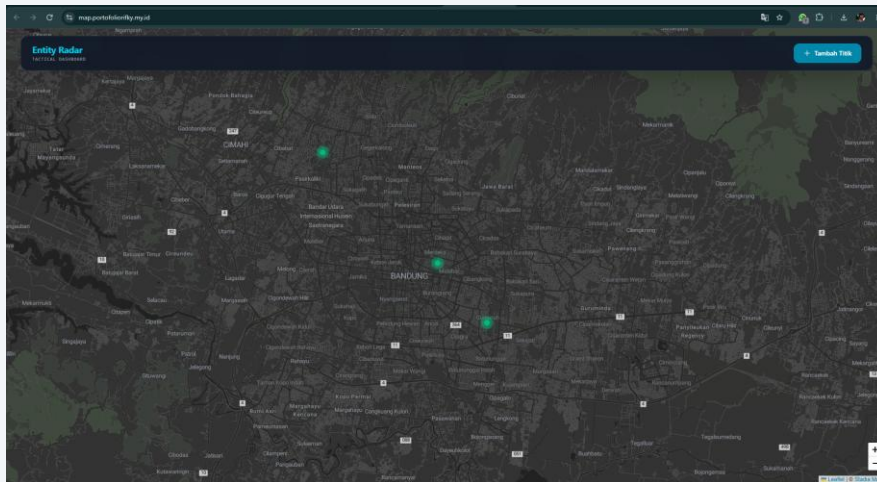
Catatan: dokumentasi endpoint di atas belum mencantumkan skema kode error (4xx/5xx) maupun autentikasi. Jika endpoint bersifat publik tanpa autentikasi, cukup nyatakan itu secara eksplisit agar tidak ambigu bagi pembaca.

4. Alur Kerja Aplikasi (User Workflow)

Bagian ini mendemonstrasikan panduan interaksi antarmuka pengguna, mulai dari peninjauan data hingga manipulasi entitas secara interaktif.

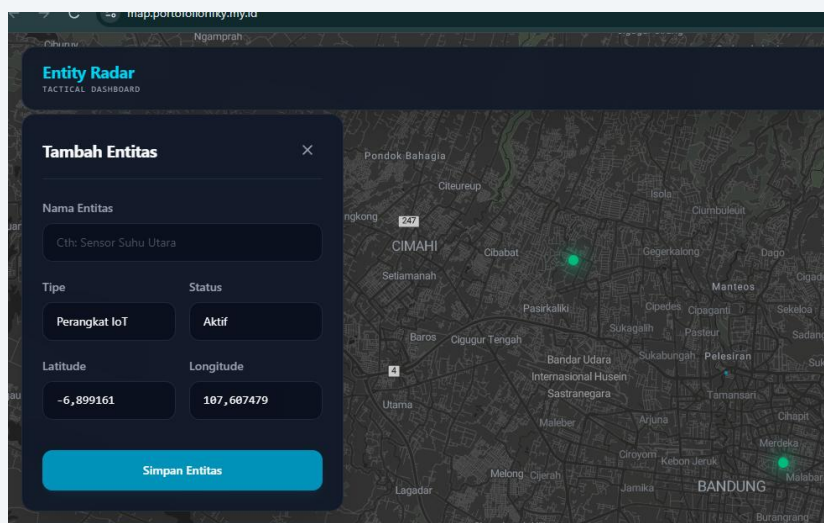
4.1 Tampilan Utama (Tactical Dashboard)

Saat aplikasi dimuat, pengguna disajikan antarmuka peta beresolusi tinggi. Titik-titik radar (marker) memvisualisasikan entitas yang ada, dengan warna denyut yang bervariasi bergantung pada status operasional entitas tersebut.



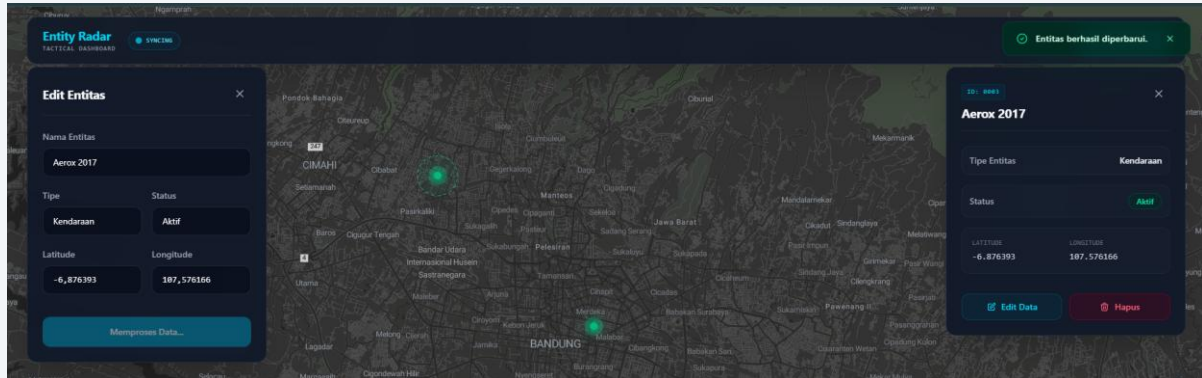
4.2 Menambahkan Entitas Baru (Geofencing Click-to-Pick)

Untuk menambahkan data, pengguna tidak perlu menginput koordinat lintang dan bujur secara manual. Pengguna cukup mengklik area mana saja di dalam kanvas peta — formulir akan otomatis terbuka dengan koordinat presisi yang telah terisi, kemudian pengguna melengkapi rincian nama, tipe, dan status.



4.3 Mengedit Data dan Sistem Notifikasi

Dengan mengklik salah satu entitas di panel kiri, pengguna dapat masuk ke mode Edit. Setiap perubahan yang dikirim akan memicu pembaruan antarmuka secara instan (zero-reload state update) dan menampilkan notifikasi toast sukses atau gagal di sudut kanan atas layar.



5. Transparansi Keterlibatan AI

Sesuai dengan prinsip integritas akademik dan profesional teknis, bagian ini merinci sejauh mana keterlibatan kecerdasan buatan (AI) dimanfaatkan selama siklus pengembangan proyek.

5.1 Ruang Lingkup Bantuan AI (Peningkatan Produktivitas)

- Perencanaan tahapan (sprint scoping): mendekomposisi spesifikasi tes mentah menjadi alur kerja iteratif.
- Scaffolding & dasar kode: mempercepat generasi boilerplate, misalnya inisialisasi modul Go, struktur konfigurasi Vite, dan draf rute Gin.
- Eksplorasi sintaks: diskusi teknis terkait manipulasi elemen SVG murni dan CSS bawaan Tailwind untuk merender radar marker kustom, mengeliminasi kebutuhan aset eksternal.

5.2 Kemandirian Pengembang (Batas Intervensi AI)

- Arsitektur logika antarmuka (UI): struktur komponen React (EntityForm, integrasi toast, mitigasi tumpang-tindih peta) direkayasa dan diselesaikan sepenuhnya secara mandiri.
- Manajemen state kritis: implementasi logika single source of truth menggunakan Zustand untuk merespons manipulasi API dilakukan murni oleh pengembang.
- Infrastruktur server (DevOps): tahapan cross-compile, resolusi kebijakan CORS dinamis, pengikatan database PostgreSQL, serta peluncuran ke reverse proxy (Cloudflare Tunnels) dieksekusi secara independen pada mesin server lokal tanpa intervensi operasional AI.

6. Keterbatasan Sistem & Evaluasi Fungsional

Sebagai wujud audit teknis, berikut cakupan yang belum diimplementasikan pada iterasi Take-Home Test ini.

6.1 Akurasi Radius Spasial (Geofencing)

Lingkaran biru pada kanvas peta mengandalkan kalkulasi manipulasi DOM visual bawaan Leaflet. Peladen belum dilengkapi algoritma analitik spasial (seperti Haversine formula) untuk melakukan pemfilteran wilayah secara presisi.

6.2 Komunikasi Dupleks (Real-Time Sync)

Modifikasi entitas dari pihak ketiga langsung melalui basis data tidak akan tercermin pada klien secara real-time. Sistem masih mengandalkan mekanisme pengambilan data saat inisialisasi ulang komponen, tanpa protokol WebSockets (WSS).

6.3 Resiliensi Mode Luring (Offline)

Belum ada integrasi Service Workers atau IndexedDB. Terputusnya koneksi jaringan akan menggagalkan siklus simpan data (drafting) karena manajemen status bersifat in-memory volatile.